

SQL DQL Übungsblatt 2 –

Fortgeschrittene Abfragen (Chinook DB)

Alle Aufgaben beziehen sich auf die Chinook-Datenbank.

Aufgaben

1. Gib alle Tracks aus, deren Länge über dem Durchschnitt aller Tracks liegt.
2. Erweitere die Ausgabe um die Differenz zur durchschnittlichen Tracklänge
3. Liste alle Kunden auf, die mehr Rechnungen erstellt haben als der Durchschnitt aller Kunden.
4. Gib die Rechnung(en) mit dem höchsten Gesamtbetrag aus.
5. Gib alle Tracks aus, die vom selben Genre sind wie der Track *Headlong*.
6. Gib zu jedem Album den zugehörigen Künstlernamen mit aus.
7. Liste alle Kunden mit Land und Anzahl der gekauften Tracks auf (nur Kunden mit Käufen).
8. Liste alle Kunden, die **keine Rechnung** haben.
9. Liste alle Rechnungen inkl. Kundennamen und deren Land.
10. Liste alle Tracks, bei denen **keine Medienart** (MediaType) hinterlegt ist.
11. Liste alle Alben auf, zu denen **keine Tracks** existieren.
12. Liste alle Tracks mit dem **Namen des Genres, Albumtitels** und des **Künstlers**.
13. Liste alle Länder auf, in denen es **mindestens zwei Kunden** gibt, die **mehr als drei Rechnungen** erstellt haben.
14. Gib alle Tracks aus, die **kürzer sind als alle anderen Tracks desselben Genres** (korrelierte Subquery).
15. Liste alle Genres mit der **durchschnittlichen Tracklänge**.
16. Gib alle Kunden mit deren **Gesamtausgaben** aus.
17. Liste alle Künstler, die **mehr als ein Album** veröffentlicht haben.
18. Liste alle Tracks in einer Liste sortiert, so dass die Tracks mit dem höchsten **Preis pro Millisekunde oben in der Liste stehen**.
19. Liste alle Kunden, die **sowohl aus den USA als auch aus Kanada** Musik gekauft haben.
20. Liste alle Rechnungen mit Kundennamen, bei denen **kein Bundesstaat** angegeben ist.
21. Listen Sie alle Kunden auf, deren E-Mail-Adresse mit 'lu' beginnt.
22. Geben Sie den Vornamen aller Mitarbeiter in Großbuchstaben und den Nachnamen in Kleinbuchstaben aus.

23. Erstellen Sie eine Liste aller Albumtitel und deren Länge (Anzahl der Zeichen) im Titel
24. Listen Sie alle Tracks auf, deren Komponist-Feld nicht NULL ist und der Komponistename das Wort 'Bach' enthält (unabhängig von Groß- und Kleinschreibung).
25. Erstellen Sie eine Liste aller Rechnungen (Invoice_Id) zusammen mit dem Jahr des Rechnungsdatums.
26. Geben Sie die 5 Rechnungen mit dem neuesten Rechnungsdatum aus, sortiert nach Rechnungsdatum (Invoice_Date) in absteigender Reihenfolge.
27. Berechnen Sie für alle Rechnungen die Dauer in Tagen vom Rechnungsdatum (Invoice_Date) bis zum aktuellen Datum. Geben Sie die Invoice_Id und die berechnete Dauer aus.
28. Finden Sie die beiden ältesten Mitarbeiter und geben Sie deren Vornamen, Nachnamen und das Alter in Jahren aus (Berechnung auf Basis von BirthDate und dem aktuellen Datum).
29. Erstellen Sie eine vereinte Liste (UNION) aller Städte (City) aus der Tabelle Customer und Employee.
30. Finden Sie alle Nachnamen (LastName) von Mitarbeitern, die keine Kunden in der Datenbank sind (EXCEPT). (Hinweis: Das Kriterium ist, dass deren Nachname nicht in der Tabelle customer enthalten ist)

Teil 2: Lösungen

1.

```
SELECT * FROM track WHERE milliseconds > (SELECT  
AVG(milliseconds) FROM track);
```

2.

```
SELECT *, Milliseconds - (SELECT AVG(Milliseconds) FROM track)  
FROM track WHERE Milliseconds > (SELECT AVG(Milliseconds) FROM  
track);
```

3.

```
SELECT customer_id FROM invoice GROUP BY customer_id HAVING  
COUNT(*) > (SELECT AVG(c) FROM (SELECT COUNT(*) AS c FROM invoice  
GROUP BY customer_id) AS sub);
```

4.

```
SELECT * FROM invoice WHERE total = (SELECT MAX(total) FROM  
invoice);
```

5.

```
SELECT * FROM track WHERE genre_id = (SELECT genre_id FROM track  
WHERE name = 'Headlong');
```

6.

```
SELECT album.title, artist.name FROM album JOIN artist ON  
album.artist_id = artist.artist_id;
```

7.

```
SELECT customer.first_name, customer.last_name, customer.country,  
COUNT(invoice_line.track_id) AS anzahl_käufe FROM customer JOIN  
invoice ON customer.customer_id = invoice.customer_id JOIN
```

```
invoice_line ON invoice.invoice_id = invoice_line.invoice_id
GROUP BY customer.customer_id, customer.first_name,
customer.last_name, customer.country;
```

Anmerkung: Aufgabe etwas unscharf formuliert. Wenn ein Kauf eine Rechnung ist, dann kein Join zur invoice-line notwendig

8.

```
SELECT customer.customer_id, customer.first_name,
customer.last_name FROM customer LEFT JOIN invoice ON
customer.customer_id = invoice.customer_id WHERE
invoice.invoice_id IS NULL;
```

Lösung auch über Subquery möglich, z.B.

```
SELECT * FROM customer AS Cus WHERE Cus.customer_id NOT IN
(SELECT Inv.customer_id from invoice Inv);
```

9.

```
SELECT invoice.invoice_id, customer.first_name,
customer.last_name, customer.country FROM invoice JOIN customer
ON invoice.customer_id = customer.customer_id;
```

10.

```
SELECT * FROM track WHERE media_type_id IS NULL;
```

11.

```
SELECT album.* FROM album LEFT JOIN track ON album.album_id =
track.album_id WHERE track.album_id IS NULL;
```

Lösung auch hier über Subquery möglich

```
SELECT * FROM album AS alb WHERE alb.album_id NOT IN (SELECT
album_id FROM track);
```

12.

```
SELECT track.name, genre.name AS genre, album.title AS album,
artist.name AS künstler FROM track JOIN genre ON track.genre_id =
```

```
genre.genre_id JOIN album ON track.album_id = album.album_id JOIN  
artist ON album.artist_id = artist.artist_id;
```

13.

```
SELECT country FROM customer WHERE customer_id IN (SELECT  
customer_id FROM invoice GROUP BY customer_id HAVING COUNT(*) >  
3) GROUP BY country HAVING COUNT(*) >= 2;
```

14.

```
SELECT * FROM track t1 WHERE milliseconds < ALL (SELECT  
milliseconds FROM track t2 WHERE t1.genre_id = t2.genre_id AND  
t1.track_id <> t2.track_id);
```

15.

```
SELECT genre.name, AVG(track.milliseconds) AS durchschnitt FROM  
genre JOIN track ON genre.genre_id = track.genre_id GROUP BY  
genre.name;
```

16.

```
SELECT customer.customer_id, SUM(invoice.total) AS gesamt FROM  
customer JOIN invoice ON customer.customer_id =  
invoice.customer_id GROUP BY customer.customer_id;
```

17.

```
SELECT artist.name FROM artist WHERE artist_id IN (SELECT  
artist_id FROM album GROUP BY artist_id HAVING COUNT(*) > 1);
```

18.

```
SELECT name, unit_price / milliseconds AS preis_pro_millis FROM  
track WHERE milliseconds > 0 ORDER BY preis_pro_millis DESC;
```

19.

```
SELECT customer.customer_id FROM customer JOIN invoice ON  
customer.customer_id = invoice.customer_id WHERE country IN  
('USA', 'Canada') GROUP BY customer.customer_id HAVING  
COUNT(DISTINCT country) = 2;
```

20.

```
SELECT invoice.invoice_id, customer.first_name,  
customer.last_name FROM invoice JOIN customer ON  
invoice.customer_id = customer.customer_id WHERE billing_state IS  
NULL;
```

21.

```
SELECT * FROM customer WHERE email LIKE 'lu%';
```

22.

```
SELECT UPPER(first_name) AS vorname_gross, LOWER(last_name) AS  
nachname_klein FROM employee;
```

23.

```
SELECT title, LENGTH(title) AS titellänge FROM album;
```

24.

```
SELECT name, composer FROM track WHERE composer IS NOT NULL AND  
LOWER(composer) LIKE '%bach%';
```

25.

```
SELECT invoice_id, EXTRACT(YEAR FROM invoice_date) AS  
rechnungsjahr FROM invoice;
```

26.

```
SELECT * FROM invoice ORDER BY invoice_date DESC LIMIT 5;
```

27.

```
SELECT invoice_id, (CURRENT_DATE - invoice_date::DATE) AS  
dauer_in_tagen FROM invoice;
```

```
-- Typumwandlung wird benötigt, da invoice_date vom Typ Timestamp  
ist
```

```
-- Alternative in Standard SQL
```

```
SELECT invoice_id, (CURRENT_DATE - CAST(invoice_date AS DATE)) AS  
dauer_in_tagen FROM invoice;
```

28.

```
SELECT first_name, last_name, EXTRACT(YEAR FROM AGE(CURRENT_DATE,  
birth_date)) AS alter_in_jahren FROM employee ORDER BY birth_date  
ASC LIMIT 2;
```

29.

```
SELECT city FROM customer UNION SELECT city FROM employee ORDER  
BY city;
```

30.

```
SELECT last_name FROM employee EXCEPT SELECT last_name FROM  
customer;
```